```
to version ${{ env.APP_VERSION }}"
          branch: main
```

❶ Executes the workflow only when a new app_delivery event is received, dispatched from another repository

❷ Saves the event payload data as environment variables for convenience

❸ Checks out the repository

❹ Navigates to the production overlay for the given application

❺ Updates the image name and version via Kustomize for the given application

❻ Updates the tag used by Kustomize to access the correct base manifests stored in the application repository

❼ An action to commit and push the changes applied to the current repository from the previous step

❽ Details about the commit operation

That's all we need for now. Commit and push the changes to your remote `polar-deployment` on GitHub. Then go back to your Catalog Service project, commit your previous changes to the acceptance stage, and push them to your remote `catalog-service` on GitHub.

The new commit to the `catalog-service` repository will trigger the deployment pipeline. First, the commit stage will produce a container image (our release candidate) and publish it to GitHub Container Registry. Then the acceptance stage will fictitiously run further tests on the application and finally send a notification (a custom `app_delivery` event) to the `polar-deployment` repository. The event triggers the production stage, which will update the production Kubernetes manifests for Catalog Service and commit the changes to the `polar-deployment` repository. Figure 15.7 illustrates the inputs and outputs for the three stages of the deployment pipeline.